QUANSER
INNOVATE·EDUCATE

Lab Procedure
Load Cell

Introduction

Ensure the following:

1. You have reviewed the Application Guide – Load Cell
2. Make sure the **Mechatronic Sensors Trainer** is out of its case and power up the **Mechatronic Sensors Trainer** by connecting the provided USB cable to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.
3. You have 3-5 small objects with known weights. They will be used to calibrate the load cell. One example is your mobile phone, by looking up the exact model, their rough weight can be found.
4. Plug in the load cell cable, make sure the red wire is on the left side of the socket.

Analyzing Raw Load Cell Data

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **`File > Open Folder`**…, and browse to the directory containing the python files and lab procedure for the Load Cell lab (…/sensors_trainer/1_fundamentals/load_cell/hardware/python). Open this directory.
2. Open load_cell_scope.py. Run this script using the ▷ button or the terminal. This will open a plot displaying raw load cell data. Gently tap the top of the load cell scale, does plot display the expected results? Stop the script using **`Ctrl+C`**.
3. In the Set-up region of the script, make sure the **duration** parameter to 5. Run the script and let the load cell stay still for 5 seconds. The mean and standard deviation of the raw data over this duration will be printed. Why is this information important?
4. Add one object to the load cell and run the scripts for 5 seconds again. How does mean and standard deviation change with the added weight?
5. Stack more objects on the load cell and repeat the process. Make sure the stacked objects does not exceed the weight limit (3 kg). What trend do you notice?

Load Cell Calibration

6. In the previous section, we have observed the trend of load cell data with increasing weight. In this section, we want to quantitatively define this trend. In the same directory, open load_cell_calibrate.py. You will need to complete the functions **process()** and **calibrate()** in the next steps.

7. As observed in the previous sections, raw data from the load cell can be noisy. In the **process()** function, we want to output the mean of the data over a period specified by **weightTime**. This function should collect raw load cell data for specified duration and return the mean. Use **sensorDataBuffer** to store and process load cell data and complete the function.

8. Run this script using the ▷ button. You will be prompted to enter the weight your selected object. After specified duration, the output of **process()** should be printed in the terminal. Verify that **process()** produces expected outputs.

9. Stop the script using **`Ctrl+C`** when you're satisfied with the output from **process()**.

10. Now we complete the **calibrate()** function. The input **recordedData** should be a list of recorded weight and voltage pairs. As observed in the previous section, the raw data from the load cell increases linearly with the weight of the object, we can assume the weight and voltage satisfy the following:

$$\text{Weight} = a \times \text{Voltage} + b$$

These parameters **a** and **b** should be stored in the variable **coefficients** and be the output of **calibrate()**. You can utilize common libraries such as Numpy's **polyfit()** and complete this function.

11. Once both functions are completed, run the script using the ▷ button. When prompted, place a known weight on the load cell and enter the weight value. Repeat this step for multiple know weights, including no weight (0 g).

12. When finished, type **'q'** or press **Enter** on an empty line to end the calibration session. The calibration coefficients are printed in the terminal for reference. Save these values for later use in the measurement section.

13. Repeat step 11 and 12 with only two weights (0 and one extra know weight) and compare the calibration coefficients. Is there significant difference with the coefficients derived from multiple weights? Which one would be more accurate?

Measuring Weight with Load Cell

14. In the same directory, open load_cell_measure.py. In this section, you will use the calibrated parameters to convert live load cell readings into corresponding weight values.

15. Fill in the **coefficient** variables with values recorded from previous sections.

16. In the main load cell data collection loop, append each new sensor reading to the **buffer** variable (a **deque** object) and implement a moving average filter to reduce signal noise. The **deque** provides an effective first-in, first-out (FIFO) data structure

which replaces the oldest value with the newest value once the maximum length is reached.

17. Use the linear calibration equation derived from the Load Cell Calibration Section to estimate the weight from the filtered voltage readings and complete the main measuring loop.

18. Run the script and put various objects on the load cell. How accurate is the estimated weight? What is the magnitude of the uncertainty? Record your observations.

19. Once all measurements are completed, power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.